Note méthodologique : preuve de concept

Projet 9 OpenClassrooms – Développez une preuve de concept (segmentation sémantique)

Dataset retenu

Le jeu de données utilisé est Cityscapes, une référence pour la compréhension sémantique de scènes urbaines. Il regroupe des images prises depuis un véhicule en circulation, avec des annotations pixel à pixel de 30 classes (19 classes principales utilisées pour l'évaluation standard).

- Données sources : leftImg8bit (images RGB) et gtFine (masques en labelIds).
- 50 villes, 5 000 images finement annotées (+ 20 000 images en annotations "coarse"), en 2048×1024 px.
- Split fin au niveau ville : 2 975 train / 500 val / 1 525 test (annotations test non publiées).

Pour itérer rapidement sur CPU, un sous-ensemble a été construit à partir des splits officiels : TRAIN sur 3 villes (aachen, bochum, bremen), VALIDATION sur frankfurt et TEST sur berlin. Le sous-ensemble d'entraînement contient 586 images et 586 masques.

Prétraitements appliqués : redimensionnement en 256×512, normalisation des images et remapping des masques de 34 labelIds vers 8 classes métier (objectif : simplifier la tâche et stabiliser l'apprentissage). Les masques sont ensuite convertis en one-hot (forme : (batch, 256, 512, 8)).

Classe	Nom	LabelIds Cityscapes
0	background	0, 1, 2, 3, 4, 5, 6
1	road	7
2	sidewalk	8
3	building	11
4	other construction	12, 13, 14, 15, 16
5	object	17, 18, 19, 20
6	vegetation	21, 22
7	vehicle	23 à 33

Figure 1 – Exemple de masque Cityscapes avant (34 labelIds) et après remapping (8 classes).

Les concepts de l'algorithme récent

1) Pourquoi remplacer VGG16 ?

Le projet part d'une baseline héritée du projet 8 : un UNet dont l'encodeur est VGG16. VGG16 est une architecture historique et très robuste, mais elle a été conçue à l'origine pour la classification et ne cherche pas l'efficacité de calcul. Sur un environnement sans GPU, ce choix limite fortement le rythme des itérations et donc la capacité à tester des variantes.

L'objectif de la preuve de concept est donc simple : conserver (voire améliorer) la qualité de segmentation tout en réduisant nettement le coût d'entraînement. Pour cela, le backbone VGG16 est remplacé par EfficientNetV2B0, un ConvNet moderne conçu pour être plus rapide à entraîner et plus efficient en paramètres.

2) EfficientNetV2 : les idées qui font la différence

EfficientNetV2 (2021) n'est pas seulement une nouvelle "famille" de modèles : c'est une démarche de conception qui vise explicitement l'efficacité d'entraînement, sans sacrifier la précision. Les points clés sont :

- Recherche d'architecture "training-aware" : la recherche automatique ne cible pas uniquement la précision, mais intègre aussi des contraintes de vitesse d'entraînement et d'efficacité en paramètres.
- Blocs Fused-MBConv : une variante plus efficace des MBConv (utilisés dans EfficientNet), utile dans les premiers étages du réseau pour mieux exploiter le matériel.
- Progressive Learning : entraînement par paliers (résolution et régularisation) pour accélérer la convergence tout en maintenant la qualité finale.

À un niveau très concret, EfficientNetV2 reste une architecture convolutionnelle. Une convolution 2D calcule une combinaison locale des pixels : $(I * K)(x, y) = \Sigma_i \Sigma_j K(i, j) \cdot I(x - i, y - j)$. L'intérêt des blocs modernes (depthwise, expansion, fusion) est d'obtenir le même type de traitement avec moins d'opérations inutiles et une meilleure utilisation du calcul.

3) Du backbone à la segmentation : EfficientNetV2 + UNet

EfficientNetV2 est initialement entraîné pour la classification. Dans ce projet, il est utilisé comme encodeur (feature extractor) : on retire la tête de classification et on conserve les cartes de caractéristiques à plusieurs niveaux de résolution. Ces cartes alimentent ensuite le décodeur de type UNet.

UNet repose sur un schéma encodeur/décodeur avec des "skip connections". L'encodeur capture le contexte global (formes, structure de la scène), tandis que le décodeur reconstruit une carte de segmentation à la résolution d'entrée. Les connexions latérales réinjectent des détails fins (bords, petits objets) pour améliorer la localisation.

Enfin, afin de rester dans un cadre comparable à la baseline, l'encodeur est gardé gelé (poids non mis à jour) et l'apprentissage se concentre sur le décodeur et la tête de segmentation. Ce choix réduit le temps de calcul et stabilise l'entraînement avec un nombre d'epochs limité.

4) Zoom sur les blocs (MBConv vs Fused-MBConv) et le choix B0

Les EfficientNet reposent sur des blocs MBConv (Mobile Inverted Bottleneck). L'idée est de séparer une convolution "coûteuse" en opérations plus légères : on élargit d'abord les canaux (expansion via 1×1), on applique une convolution depthwise (par canal), puis on

reprojette les canaux (1×1). Cela réduit fortement le nombre d'opérations, tout en conservant de bonnes capacités de représentation.

- MBConv : très efficace en paramètres, mais pas toujours optimal en vitesse selon le matériel (overhead des depthwise, accès mémoire).
- Fused-MBConv : fusion partielle des opérations (convolution standard au début du bloc), souvent plus rapide dans les premiers étages où la résolution est élevée.
- Conséquence pratique : EfficientNetV2 vise une meilleure vitesse d'entraînement "réelle", pas seulement une meilleure efficacité théorique.

Dans la famille EfficientNetV2, le choix B0 est cohérent avec l'objectif du projet : un backbone compact, simple à intégrer (Keras/TensorFlow), et suffisamment performant pour servir d'encodeur UNet. Des variantes plus grandes (B1–B3, puis S/M/L) peuvent améliorer la précision, mais elles augmentent le coût et s'éloignent du scénario "itération rapide / ressources limitées".

5) Progressive Learning : pourquoi ça compte en pratique

EfficientNetV2 popularise aussi une stratégie d'entraînement progressive : au lieu de tout activer dès le début (grande résolution, augmentations fortes, dropout important), on démarre avec une configuration plus simple, puis on augmente progressivement la difficulté. L'intérêt est double :

- Gagner du temps au début de l'entraînement (images plus petites / régularisation plus faible).
- Stabiliser l'optimisation : le modèle apprend d'abord les grandes structures, puis affine les détails.
- Mieux exploiter un budget d'epochs limité : on évite de "payer" trop tôt un coût de calcul maximal.

Dans ce projet, même si l'entraînement reste volontairement court, l'ADN d'EfficientNetV2 (architecture + stratégie) se retrouve dans la dynamique observée : convergence rapide et coût par epoch faible, ce qui rend l'approche plus compatible avec des contraintes de production ou de prototypage.

6) Points clés à retenir pour la POC

En pratique, EfficientNetV2B0 apporte surtout un meilleur rapport qualité/temps d'entraînement, ce qui est idéal pour itérer rapidement sur CPU.

Mécanisme	Impact (POC)
Fused-MBConv	Accélère les premiers étages → gain de temps.
Training-aware design	Compromis précision/vitesse optimisé.
Progressive Learning	Converge vite avec peu d'epochs.
B0 compact	Bon ratio performance / coût (CPU/embarqué).

Cela permet de tester plus de variantes (loss, augmentations, fine-tuning) à budget temps constant.

La modélisation

1) Pipeline de données

Le pipeline reprend l'approche du projet 8, en l'adaptant à un entraînement plus rapide : lecture des images et masques, redimensionnement en 256×512, normalisation des images, puis remapping des labels Cityscapes vers 8 classes. Le dataloader renvoie des batches cohérents : X en (batch, 256, 512, 3) et y en (batch, 256, 512, 8) après encodage one-hot.

Figure 2 – Exemple d'image normalisée et de masque remappé (8 classes).

Le remapping est réalisé via une table de correspondance (extrait ci-dessous). L'idée est de regrouper les classes fines (labelIds) en classes plus "métier" et plus stables pour une preuve de concept.

```
CITYSCAPES_34_TO_8 = {
    0:0, 1:0, 2:0, 3:0, 4:0, 5:0, 6:0,      # background
    7:1,                                    # road
    8:2,                                    # sidewalk
    11:3,                                   # building
    12:4, 13:4, 14:4, 15:4, 16:4,           # other construction
    17:5, 18:5, 19:5, 20:5,                 # object
    21:6, 22:6,                             # vegetation
    23:7, 24:7, 25:7, 26:7, 27:7, 28:7,
    29:7, 30:7, 31:7, 32:7, 33:7            # vehicle
}
```

Une fois les masks remappés, la dernière étape consiste à produire une sortie compatible avec l'entraînement : un tenseur one-hot sur 8 canaux, associé à une softmax en sortie du modèle.

2) Modèles comparés et protocole d'entraînement

Deux architectures sont entraînées strictement dans les mêmes conditions, afin d'isoler l'impact du backbone : UNet + VGG16 (baseline) et UNet + EfficientNetV2B0 (algorithme récent). Dans les deux cas, l'encodeur pré-entraîné est gelé et seul le décodeur UNet est appris.

Paramètre	Valeur
Tâche	Segmentation sémantique multi-classes (8 classes)
Entrées	Images 256×512×3 (après redimensionnement)
Batch size	4
Époques	7 (même durée pour les deux modèles)
Batches/epoch	145
Environnement	CPU (pas de GPU)
Encodeur	Pré-entraîné et gelé

Ce protocole est volontairement simple : l'objectif n'est pas d'atteindre l'état de l'art sur Cityscapes, mais de vérifier qu'un backbone plus moderne permet d'obtenir une segmentation au moins équivalente, avec un coût expérimental nettement plus faible.

D'un point de vue architectural, les deux modèles partagent le même principe : les features extraites à différentes échelles par l'encodeur alimentent le décodeur via des skip connections (concaténation des features de bas niveau avec les features upsamplées). La tête de segmentation projette ensuite les features finales en 8 canaux, suivis d'une softmax pour obtenir une distribution de probabilité par pixel.

Le choix de geler le backbone répond à deux contraintes : (1) limiter le temps de calcul sur CPU ; (2) éviter un surapprentissage rapide sur un sous-ensemble de villes. Dans une version "production", ce choix serait réévalué avec un fine-tuning contrôlé.

3) Métriques et score synthétique

Pour évaluer la qualité de segmentation (A : masque réel, B : masque prédit), trois métriques complémentaires sont suivies sur le jeu de validation. La loss est également surveillée pour suivre la convergence.

- Dice = $2 \cdot |A \cap B| / (|A| + |B|)$
- IoU = $|A \cap B| / |A \cup B|$
- Pixel accuracy = (pixels correctement prédits) / (pixels totaux)
- Score synthétique = moyenne(Dice, IoU, pixel accuracy)

Principe d'architecture : scaling conjoint (EfficientNetV2)

EfficientNetV2 s'inscrit dans la continuité d'EfficientNet : la complexité globale du réseau est pilotée par un scaling conjoint. Formulation simplifiée : Complexité ≈ profondeur × largeur × résolution. L'intérêt est de répartir le budget de calcul entre ces trois leviers plutôt que d'en surdimensionner un seul.

Levier	Ce qui varie	Effet principal	Lecture POC
Profondeur	Nombre de blocs/couches	Capacité de représentation ↑	Utile si sous-apprentissage, mais coût CPU ↑
Largeur	Nombre de canaux	Features plus riches	Augmente mémoire/compute, à calibrer
Résolution	Taille d'entrée (H×W)	Détails spatiaux ↑	Coût augmente vite : compromis (256×512)

Dans cette preuve de concept, le choix d'EfficientNetV2B0 correspond à un niveau de scaling compact, adapté à l'entraînement sur CPU : il limite le temps par epoch tout en conservant une extraction de features suffisamment riche pour le décodeur UNet.

4) Démarche d'optimisation (dans le cadre de la POC)

La démarche d'optimisation suivie ici est volontairement minimaliste : on fixe un cadre identique pour les deux architectures (mêmes données, même taille d'entrée, même nombre d'epochs), puis on compare (1) la performance finale et (2) le coût expérimental. L'idée est de valider rapidement le "bon sens" du remplacement de backbone, avant d'engager une phase plus longue d'ajustements (fine-tuning, augmentations, loss).

Une synthèse des résultats

1) Résultats finaux sur le jeu de validation

Les deux modèles sont entraînés 7 epochs dans un environnement CPU, avec un pipeline identique. Le tableau ci-dessous synthétise les performances finales observées sur la validation.

Modèle	Loss	Dice	IoU	Pixel acc.	Temps/epoch
UNet + VGG16	≈ 0.41	≈ 0.59	≈ 0.22	≈ 0.59	≈ 230–243 s
UNet + EfficientNetV2B0	≈ 0.38	≈ 0.62	≈ 0.23	≈ 0.62	≈ 54–61 s

Lecture rapide : les métriques de segmentation sont très proches, mais EfficientNetV2B0 garde un léger avantage sur Dice, IoU et pixel accuracy. Le gain le plus marqué concerne le temps d'entraînement : environ 4× plus rapide par epoch dans les mêmes conditions CPU.

Pour compléter la lecture, le score synthétique (moyenne Dice/IoU/pixel accuracy) passe : • VGG16 : ≈ 0.12 → ≈ 0.47 ; • EfficientNetV2B0 : ≈ 0.14 → ≈ 0.49. Les deux modèles progressent fortement, avec un léger avantage final pour EfficientNetV2B0.

2) Avant / après entraînement : progression des métriques

Les graphiques suivants comparent les métriques de validation avant et après apprentissage. Ils permettent de vérifier que la progression est cohérente sur l'ensemble des indicateurs.

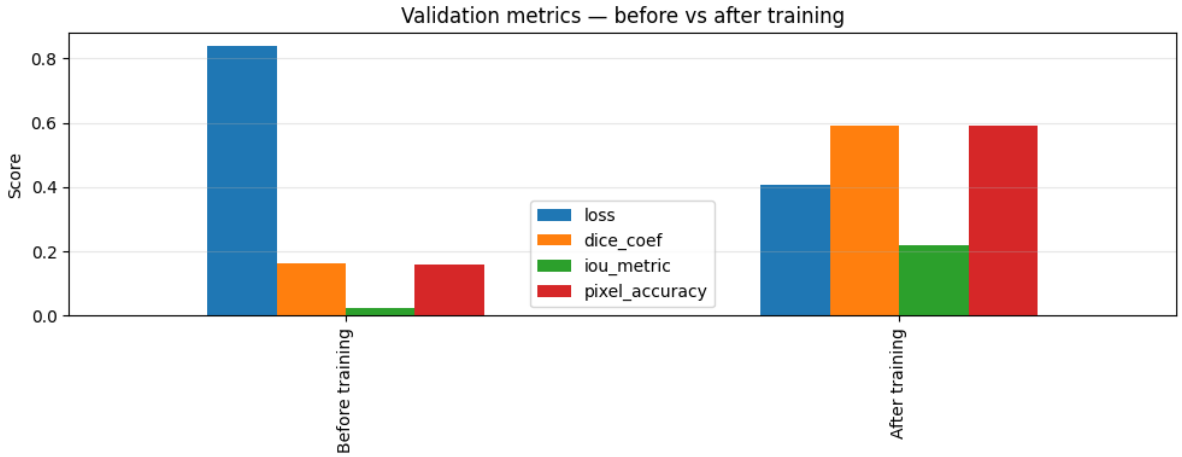

Figure 3 – UNet + VGG16 : métriques de validation avant vs après entraînement.

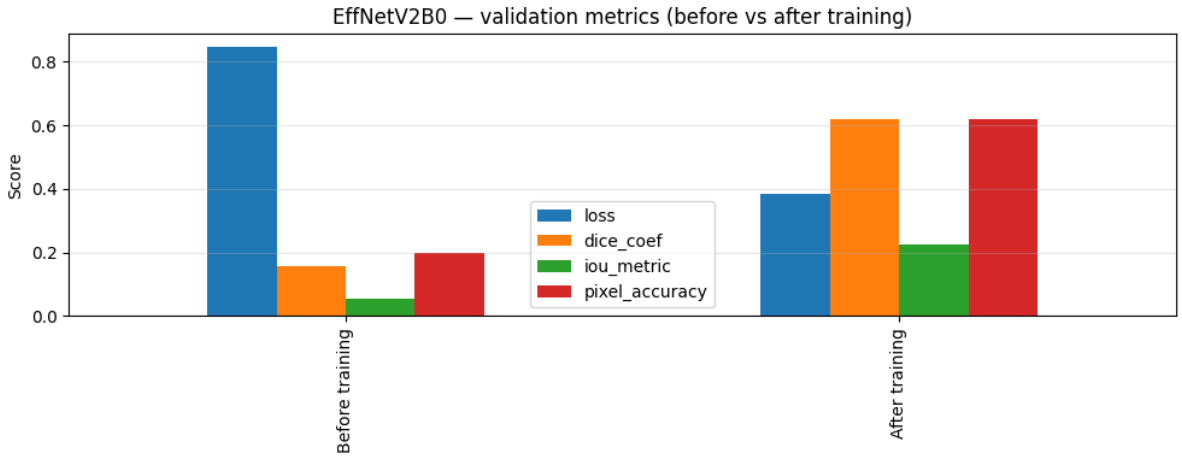

Figure 4 – UNet + EfficientNetV2B0 : métriques de validation avant vs après entraînement.

Dans les deux cas, l'entraînement transforme une baseline initialement peu exploitable en un modèle fonctionnel. Les gains sont visibles sur Dice, IoU et pixel accuracy, tandis que la loss diminue. EfficientNetV2B0 converge légèrement plus vite et atteint une performance finale un peu supérieure, avec un temps d'itération beaucoup plus favorable.

3) Conclusion de la preuve de concept

La preuve de concept valide l'hypothèse de départ : remplacer VGG16 par un backbone plus récent permet de garder une qualité de segmentation comparable, tout en rendant l'entraînement nettement plus rapide sur CPU. Dans un contexte de production ou d'itération rapide (tests d'hyperparamètres, ablations, nouvelles augmentations), EfficientNetV2B0 apparaît comme un choix plus pertinent.

À retenir : le gain de vitesse n'est pas un "détail" — il change directement le cycle de développement. Un modèle 4× plus rapide à entraîner permet de tester plus d'idées (augmentations, choix de loss, taille d'image) à budget temps constant. C'est exactement le type de bénéfice recherché dans une preuve de concept orientée déploiement embarqué ou ressources limitées.

4) Résumé des gains

À performances proches, EfficientNetV2B0 réduit fortement le temps d'entraînement, ce qui accélère le cycle d'itération.

Indicateur	**VGG16**	**EffNetV2B0**
Score synthétique	≈ 0,47	≈ 0,49
Dice / IoU / Acc.	≈ 0,59 / 0,22 / 0,59	≈ 0,62 / 0,23 / 0,62
Temps / epoch	≈ 230–243 s	≈ 54–61 s
Gain vitesse	—	≈ 4×

Dans une logique embarquée ou ressources limitées, ce gain de vitesse est souvent plus décisif qu'un petit gain de métriques.

L’analyse de la feature importance globale et locale du nouveau modèle

1) Pourquoi parler de “feature importance” en segmentation ?

Sur un modèle de segmentation, la prédiction n’est pas un simple label, mais une carte dense (une classe par pixel). L’interprétabilité se traite donc à deux niveaux : (1) globalement, comprendre quelles structures dominent l’apprentissage (grandes classes, textures, biais du dataset) ; (2) localement, expliquer une prédiction sur une image donnée (quelles zones ont influencé une classe).

2) Analyse globale : tendances et biais observables

Dans Cityscapes, certaines classes occupent mécaniquement une grande partie des pixels (route, bâtiments, végétation), tandis que d’autres sont plus rares et plus difficiles (objets fins, “other construction”, trottoirs selon la scène). Même si la métrique globale est correcte, cette distribution peut conduire à un modèle qui “privilégie” les classes dominantes.

- Indicateurs utiles (recommandés) : IoU/Dice par classe, matrice de confusion par classe, et suivi des erreurs sur les classes rares.
- Lecture qualitative : vérifier les frontières (bords route/trottoir), et la capacité à détecter des objets petits (poteaux, panneaux, véhicules lointains).

À défaut d’une batterie complète de métriques par classe, une première lecture consiste à comparer, image par image, la proportion de pixels réels vs prédits par classe. Les deux figures suivantes illustrent ce diagnostic sur deux prédictions aléatoires du modèle EfficientNetV2B0.

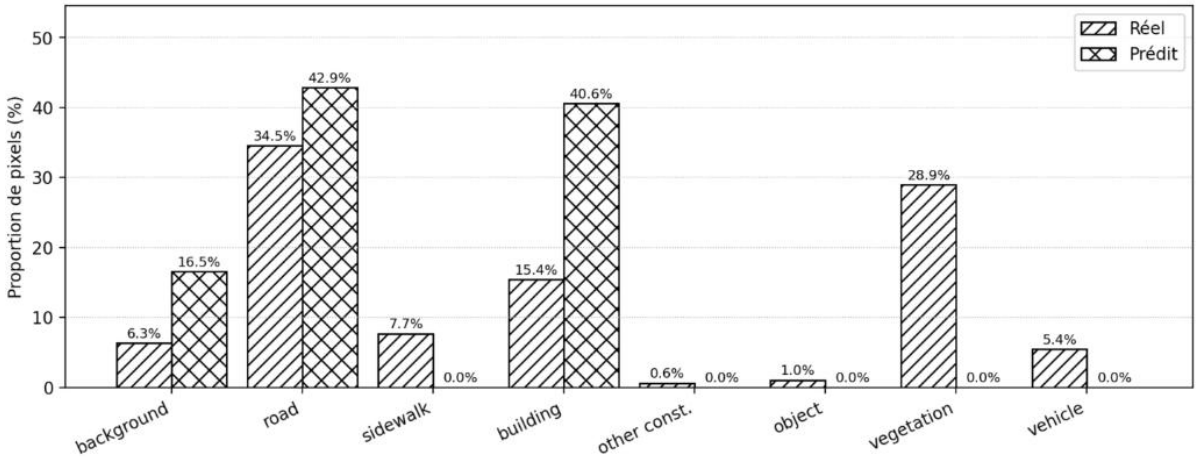

Figure 5 – Répartition des pixels par classe (réel vs prédit) – échantillon 1.

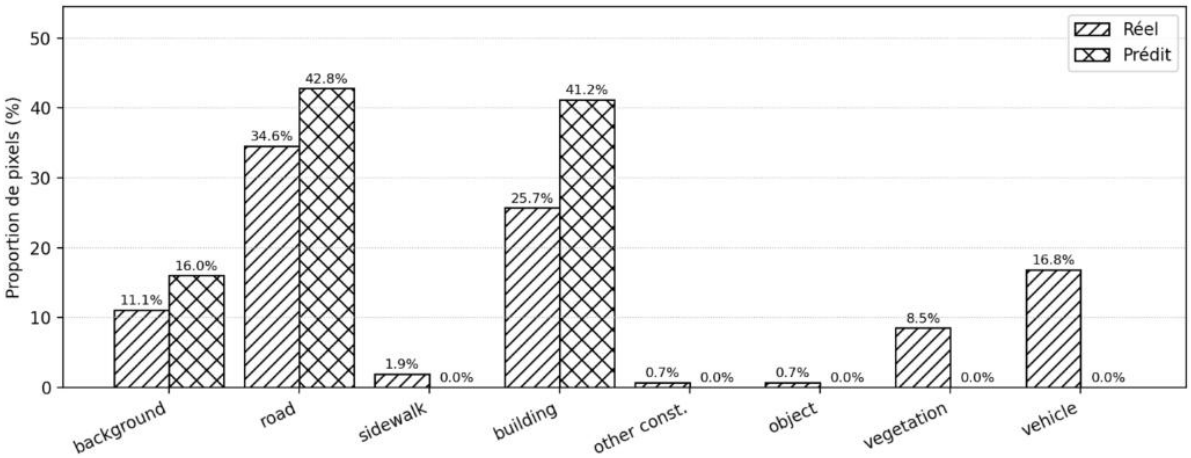

Figure 6 – Répartition des pixels par classe (réel vs prédit) – échantillon 2.

Ces graphiques montrent un comportement classique en segmentation : les classes majeures (route, bâtiment) tendent à être plus facilement détectées, parfois au détriment de classes plus fines. On observe aussi que certaines classes rares peuvent être sous-prédites (voire absentes) sur un échantillon donné, ce qui est cohérent avec une résolution réduite (256×512) et un entraînement court (7 epochs).

3) Analyse locale : expliquer une prédiction image par image

Pour une analyse locale, l'objectif est de visualiser quelles zones de l'image activent le modèle pour une classe donnée (par exemple "road" ou "vehicle"). Plusieurs méthodes sont adaptées aux CNN :

- Occlusion sensitivity : masquer une zone (patch) et mesurer l'impact sur la probabilité de la classe, pour repérer les régions critiques.
- Grad-CAM : utiliser le gradient de la classe vers la dernière couche convolutionnelle pour produire une carte d'activation (heatmap) des zones importantes.
- Integrated Gradients : attribuer l'importance à chaque pixel via une intégration des gradients entre une image de référence et l'image réelle.

Exemple (Grad-CAM, formulation simplifiée) : on calcule des poids α_k^c comme la moyenne spatiale des gradients $\partial y^c/\partial A_k$ (où A_k est la carte de caractéristiques k, et y^c le score de la classe c). La heatmap est ensuite obtenue par $\mathrm{ReLU}(\Sigma_k \alpha_k^c \cdot A_k)$. Sur une scène urbaine, on s'attend à voir des activations fortes sur les bords de route, la silhouette des véhicules et les zones à fort contraste (marquages au sol, contours).

Les limites et les améliorations possibles

Limites identifiées

Cette preuve de concept est volontairement contrainte pour permettre une comparaison rapide sur CPU. Ces choix ont des impacts directs sur les performances atteignables et sur l'interprétation des résultats :

- Sous-ensemble Cityscapes (peu de villes) : risque de biais géographique et de scènes moins variées que le dataset complet.
- Redimensionnement 256×512 : perte de détails, surtout pour les objets fins (poteaux, panneaux, piétons lointains).
- Backbone gelé et entraînement court (7 epochs) : limite la capacité d'adaptation du backbone au domaine "route".
- Métriques globales : un bon score global peut masquer une faiblesse sur les classes rares (d'où l'intérêt d'un suivi par classe).

Améliorations possibles

Les pistes suivantes sont cohérentes avec l'objectif initial (efficacité + qualité) et peuvent être activées progressivement :

- Fine-tuning : dégeler les derniers blocs de l'encodeur avec un faible learning rate, puis entraîner plus longtemps (scheduler).
- Gestion du déséquilibre : loss pondérée par classe, focal loss ou ajout d'un Dice loss, suivi IoU/Dice par classe.
- Augmentations ciblées : variations de luminosité/contraste, flou léger, random crop (en conservant la géométrie), pour mieux généraliser.
- Architecture : tester un décodeur plus riche (UNet++, attention) ou une approche type DeepLabV3+ avec backbone EfficientNetV2.
- Déploiement embarqué : quantification (TFLite), pruning et mesures de latence, afin de valider le gain réel en production.
- Interprétabilité : produire des heatmaps Grad-CAM/Integrated Gradients sur des succès/échecs, et documenter des cas limites (mauvais éclairage, pluie, scènes atypiques).

Références principales

- Cordts et al., 2016 – The Cityscapes Dataset for Semantic Urban Scene Understanding (CVPR).
- Tan & Le, 2021 – EfficientNetV2: Smaller Models and Faster Training (ICML).
- Ronneberger et al., 2015 – U-Net: Convolutional Networks for Biomedical Image Segmentation.
- Simonyan & Zisserman, 2014 – Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG).
- Selvaraju et al., 2017 – Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.
- Sundararajan et al., 2017 – Integrated Gradients (Axiomatic Attribution for Deep Networks).